

PesaSwap — Complete Test Plan

Application Overview

Stack: React 19 + TanStack Start + Vite 7 + Tailwind CSS 4 + Cloudflare Workers
Routes: `/` (Merchant App), `/pay` (Customer Tap&Go), `/table` (Customer Table Payment)
Storage: localStorage (tables, catalogue, invoices)

MODULE 1: Merchant Home & Navigation

Features

- Bottom nav with 6 tabs: TAP&GO, TABLES, HOME, INVOICE, AI, LEDGER
- Home dashboard with stats, action tiles
- Currency switcher (USD, EUR, GBP, NGN)

#	Test Type	Test Case	Expected Result
1.1	Smoke	Load app at <code>/</code>	Merchant app renders with bottom nav, no console errors
1.2	Smoke	Tap each nav tab	Each tab renders its respective view
1.3	Unit	<code>Tab</code> type allows only valid values	TypeScript enforces: home, invoice, scan, list, insights, wallets, tapgo, tables
1.4	Unit	Currency switcher state	Switching currency updates displayed balances
1.5	UAT	Navigate between all 6 tabs	Smooth transitions, active state highlighted, no flicker
1.6	UAT	App loads on mobile viewport (375px)	Full responsive layout, no overflow, nav icons readable

MODULE 2: Invoice Management

Features

- Create invoice (amount, currency, client, due date, line items)
- Recurring invoices (Weekly/Bi-weekly/Monthly)
- Partial payments & installment plans
- FX rate locking (48h)
- Invoice list with filters (All/Paid/Pending/Overdue)
- Invoice detail sheet with timeline
- QR code generation for payment
- Share via WhatsApp, SMS, Copy Link
- Batch actions (select multiple, export)
- Payment recording with method selector

#	Test Type	Test Case	Expected Result
2.1	Smoke	Open Invoice Creator	Form renders with all fields (amount, currency, client, due date)
2.2	Smoke	Create basic invoice	Invoice appears in list with status "Pending"
2.3	Unit	<code>generateInstallments()</code> with 3 monthly	Returns 3 installments with correct amounts and dates
2.4	Unit	<code>lockFxRate("USD", "KES", 48)</code>	Returns FxLock with rate, expiry 48h from now
2.5	Unit	<code>fxLockTimeRemaining(lock)</code>	Returns human-readable "Xh Ym" string
2.6	Unit	<code>totalPaid(invoice)</code>	Sums all partial payments correctly

#	Test Type	Test Case	Expected Result
2.7	Unit	<code>amountRemaining(invoice)</code>	Returns invoice.amount - totalPaid
2.8	Unit	<code>whatsAppLink(invoice, url)</code>	Returns valid wa.me URL with encoded message
2.9	Unit	<code>smsLink(invoice, url)</code>	Returns valid sms: URI
2.10	Unit	<code>payloadFor(invoice)</code>	Returns base64-encoded JSON with till, amount, merchant
2.11	Unit	<code>payLink(invoice)</code>	Returns URL with ?tapgo= parameter
2.12	Unit	<code>nextRecurringDate("Weekly")</code>	Returns date 7 days from now
2.13	Unit	<code>nextRecurringDate("Monthly")</code>	Returns date 1 month from now
2.14	Unit	<code>timeAgo("2026-05-30T10:00:00Z")</code>	Returns correct relative time string
2.15	Unit	Invoice list filter "Paid"	Only shows invoices with status "Paid"
2.16	Unit	Invoice list filter "Overdue"	Only shows invoices past due date and unpaid
2.17	UAT	Create invoice → share via WhatsApp	WhatsApp opens with pre-filled message containing pay link
2.18	UAT	Create invoice → record partial payment	Balance updates, timeline shows event, status changes to "Partial"
2.19	UAT	Create recurring invoice	Auto-generates next invoice on schedule
2.20	UAT	Lock FX rate → view countdown	Timer counts down, rate stays locked until expiry
2.21	UAT	Export multiple invoices	CSV/JSON file downloads with correct data
2.22	UAT	View invoice detail → copy link	Clipboard contains valid payment URL

MODULE 3: Tap & Go POS (Merchant)

Features

- Numpad entry for amount
- QR code generation (encodes till + amount + merchant)
- Option for customer phone entry (STK push)
- Payment simulation (waiting → success)
- Receipt display with transaction ID

#	Test Type	Test Case	Expected Result
3.1	Smoke	Navigate to TAP&GO tab	Numpad renders with KES label
3.2	Smoke	Enter amount → Generate QR	QR code appears with encoded payment data
3.3	Unit	Amount entry validation	Only numeric, max reasonable amount, no leading zeros
3.4	Unit	QR payload encoding	<code>base64(JSON{till, amount, merchant})</code> matches expected
3.5	Unit	STK push phone validation	Accepts 07XX/01XX formats, rejects invalid
3.6	UAT	Full flow: Enter 500 → QR → Customer scans → Payment received	Success screen with amount, receipt ID, notification sound
3.7	UAT	Enter phone number → STK push flow	Simulated push notification, success confirmation

MODULE 4: Customer Pay Page (/pay)

Features

- QR scan or manual entry
- Payment data decode from URL param
- Confirm screen (merchant, amount, till)
- Customer phone number input
- PIN entry (6 digits) or biometric toggle

- Processing animation → Success
- Receipt display

#	Test Type	Test Case	Expected Result
4.1	Smoke	Load <code>/pay</code> without params	Shows idle state with scan/enter options
4.2	Smoke	Load <code>/pay?tapgo=<valid_base64></code>	Auto-decodes and shows confirm screen
4.3	Unit	Decode invalid base64	Gracefully shows error, doesn't crash
4.4	Unit	PIN input accepts exactly 6 digits	Blocks after 6, only numeric
4.5	Unit	Biometric toggle state	Switches between PIN pad and biometric prompt
4.6	Unit	Phone number format validation	Validates Kenyan mobile format
4.7	UAT	Full payment flow: Scan → Confirm → PIN → Success	All states transition correctly, success shows receipt
4.8	UAT	Biometric flow: Scan → Confirm → Biometric → Success	Biometric prompt appears, success on confirm
4.9	UAT	Mobile viewport (375px)	All elements accessible, keyboard doesn't overlap

MODULE 5: Table Service (Merchant)

Features

- Create/manage tables with number & server assignment
- Table states: open → requesting-bill → partially-paid → closed
- Add menu items to table
- Quick Charge (amount-only tables)
- Table overview with stats (Active, Revenue, Tips)
- Table detail view with item list, payments, actions
- QR generation per table
- Auto-close when fully paid
- Walkout risk alert (2h+ open, \$0 paid)
- Real-time staff notifications (toast + audio beep)
- Close table manually
- Refund flow per payment

#	Test Type	Test Case	Expected Result
5.1	Smoke	Navigate to TABLES tab	Overview renders with stats and action buttons
5.2	Smoke	Create new table (number 5, server "Grace")	Table appears in active list
5.3	Unit	<code>getTotal(table)</code> with items	Sums (price × qty) for all items
5.4	Unit	<code>getTotal(table)</code> with quickCharge	Returns quickCharge amount
5.5	Unit	<code>getRemainingBalance(table)</code>	Returns total - paidAmount
5.6	Unit	<code>createTable()</code> with empty number	Validation prevents creation
5.7	Unit	<code>createTable()</code> with duplicate number	Validation prevents duplicate
5.8	Unit	<code>notifyStaff(tableNum, amount, payer)</code>	Fires toast and plays audio beep
5.9	Unit	Auto-close: <code>paidAmount >= total</code>	Status changes to "closed", <code>closedAt</code> set
5.10	Unit	Walkout detection: open 2h+, \$0 paid	Table flagged in <code>walkoutRiskTables</code>
5.11	Unit	<code>generateTableQR(table)</code>	Returns URL with valid base64 payload
5.12	UAT	Full table lifecycle: Create → Add items → Customer pays → Auto-close	All transitions smooth, notifications fire
5.13	UAT	Quick Charge: Create → Enter 1000 → Generate QR → Payment	Quick charge table created, paid, closed
5.14	UAT	Refund payment	Confirm dialog → payment removed → balance updated
5.15	UAT	Walkout alert visible	Red pulsing alert shows for tables 2h+ unpaid

#	Test Type	Test Case	Expected Result
5.16	UAT	Multiple tables simultaneously	Each operates independently, no state bleed

MODULE 6: Catalogue Management (Merchant)

Features

- CRUD for menu items (name, price, category)
- Categories: Main, Side, Drink, Dessert
- Item count displayed
- localStorage persistence

#	Test Type	Test Case	Expected Result
6.1	Smoke	Open Catalogue view	Lists all items with edit/delete buttons
6.2	Unit	<code>saveCatalogueItem()</code> — new item	Adds to catalogue array, persists to localStorage
6.3	Unit	<code>saveCatalogueItem()</code> — edit existing	Updates item in place
6.4	Unit	<code>deleteCatalogueItem(id)</code>	Removes from array and localStorage
6.5	Unit	Validation: empty name	Prevents save
6.6	Unit	Validation: price ≤ 0	Prevents save
6.7	UAT	Add item → verify in "Add Items" table flow	New item selectable when adding to table
6.8	UAT	Delete item → refresh page	Item gone, catalogue persists

MODULE 7: Customer Table Payment (/table)

Features

- QR decode with table data (items, merchant, till, server)
- Bill view with itemized list
- Split payment options: Full, Equal split, By item, Custom amount
- Tip selection (0%, 5%, 10%, 15%, 20%, custom)
- Smart tip suggestion ("Most guests tip 10-15%")
- Round-up suggestion
- Multi-payment method (M-Pesa, Card, Apple Pay, Google Pay)
- Phone number input
- Processing → Success
- Post-payment review prompt (star rating → Google review)
- Digital receipt sharing (Web Share API / clipboard)
- Quick charge support (hides item list, hides by-item split)

#	Test Type	Test Case	Expected Result
7.1	Smoke	Load <code>/table?t=<valid_base64></code>	Decodes and shows bill with items
7.2	Smoke	Load <code>/table</code> without params	Shows error/scan prompt
7.3	Unit	Decode quickCharge table	Hides item list, hides "by-item" split option
7.4	Unit	Equal split calculation (4 people)	Total ÷ 4, rounded to nearest integer
7.5	Unit	By-item split selection	Only selected items summed
7.6	Unit	Custom amount validation	Must be > 0 and $\leq$ remaining balance
7.7	Unit	Tip % calculation (10% of 2630)	Returns 263
7.8	Unit	Round-up suggestion (2893 → 2900)	Suggests +7

#	Test Type	Test Case	Expected Result
7.9	Unit	Round-up suggestion (3000)	No suggestion shown
7.10	Unit	Review star rating state (1-5)	Updates correctly, truncated to integer
7.11	UAT	Full flow: Bill → Full pay → 10% tip → M-Pesa → PIN → Success	All 6 states transition correctly
7.12	UAT	Split equally (3 people) → pay share	Amount = total/3, tip applies to share only
7.13	UAT	Pay by item → select 2 items → tip → pay	Subtotal = selected items only
7.14	UAT	Custom amount → enter 500 → pay	Only 500 charged
7.15	UAT	Smart tip nudge visible	"Most guests tip 10-15%" banner shows
7.16	UAT	Round-up tap → adds to tip	Total adjusts by round-up amount
7.17	UAT	Post-payment: rate 5 stars	Redirects to Google review after 2s
7.18	UAT	Post-payment: share receipt	Web Share dialog opens (or copies to clipboard)
7.19	UAT	Switch payment method to "Card"	Card icon shown, flow continues
7.20	UAT	Mobile viewport (375px) entire flow	No overflow, keyboard-friendly, scrollable

MODULE 8: AI Insights (Merchant)

Features

- Customer scoring (payment reliability)
- Payment predictions
- Cash flow forecast
- Smart chase automation (overdue invoice follow-up)
- Collection risk assessment

#	Test Type	Test Case	Expected Result
8.1	Smoke	Navigate to AI tab	Insights dashboard renders
8.2	Unit	<code>computeCustomerScores(invoices)</code>	Returns scores 0-100 based on payment history
8.3	Unit	<code>computePredictions(invoices, scores)</code>	Returns predicted payment dates and amounts
8.4	Unit	<code>computeCashFlowForecast(invoices, predictions)</code>	Returns 30-day forecast array
8.5	Unit	<code>getChaseStatus(overdueInvoice)</code>	Returns correct step and next action
8.6	UAT	View customer reliability scores	Ranked list with color-coded scores
8.7	UAT	View cash flow forecast chart	Bar/line chart shows projected inflows
8.8	UAT	Click overdue invoice → chase suggestion	Shows recommended next action

MODULE 9: Wallet & Reconciliation (Merchant)

Features

- Multi-currency wallet balances
- Transaction history generated from invoices
- Settlement via Wise integration

#	Test Type	Test Case	Expected Result
9.1	Smoke	Navigate to LEDGER tab	Wallet balances and transaction list render
9.2	Unit	<code>generateWalletTransactions(invoices)</code>	Generates correct transaction entries from paid invoices
9.3	UAT	View wallet with multiple currencies	All balances displayed with correct symbols

#	Test Type	Test Case	Expected Result
9.4	UAT	"Settle via Wise" button	Opens settlement flow

MODULE 10: Scan & Decode

Features

- QR/barcode text input decode
- Paste from clipboard
- Decode invoice payload from base64

#	Test Type	Test Case	Expected Result
10.1	Smoke	Navigate to Scan tab	Input field renders with paste button
10.2	Unit	<code>tryDecode(validBase64)</code>	Returns decoded invoice data
10.3	Unit	<code>tryDecode(invalidString)</code>	Returns null gracefully
10.4	Unit	<code>handlePaste()</code>	Reads clipboard and calls tryDecode
10.5	UAT	Paste valid QR text → opens invoice	Invoice detail sheet opens with correct data

MODULE 11: Tips Analytics (Merchant)

Features

- Server leaderboard (ranked by tips)
- Average tip rate calculation
- Recent tips list
- Tip totals

#	Test Type	Test Case	Expected Result
11.1	Smoke	Click "Tips Analytics" button	View renders with leaderboard
11.2	Unit	Tip rate calculation (tips/revenue × 100)	Correct percentage
11.3	Unit	Server ranking by tip total	Sorted descending
11.4	UAT	After 3 payments with tips → view analytics	All 3 servers listed, amounts correct
11.5	UAT	Zero tips scenario	Shows "No tips yet" state

MODULE 12: Payment History (Merchant)

Features

- All payments listed chronologically
- Filter by table number
- Total revenue display

#	Test Type	Test Case	Expected Result
12.1	Smoke	Click "History" button	View renders with payment list
12.2	Unit	Filter by table number 3	Only table 3 payments shown
12.3	Unit	Clear filter	All payments shown

#	Test Type	Test Case	Expected Result
12.4	Unit	Total calculation	Sum of (amount + tip) for filtered results
12.5	UAT	10 payments across 3 tables → filter each	Correct subset displayed each time

MODULE 13: Intelligence Layer — Revenue Forecast

Features

- Daily average revenue calculation
- Weekly projection (avg × 7)
- Trend indicator (up/down/flat)
- Revenue by day bar chart
- AI insights (peak hour, avg spend/table, growth targets)

#	Test Type	Test Case	Expected Result
13.1	Smoke	Click "Revenue Forecast"	View renders with charts and insights
13.2	Unit	Daily average: total ÷ active days	Correct calculation
13.3	Unit	Trend: projected > actual → "up"	Shows 1 indicator
13.4	Unit	Peak hour detection	Finds hour with highest revenue
13.5	Unit	Growth target calc: (projected×1.2 - projected) ÷ avgPerTable	Correct additional tables needed
13.6	UAT	With 7 days of data → view forecast	All 7 bars rendered, insights contextual
13.7	UAT	With 0 payments → view forecast	Graceful empty state, no divide-by-zero

MODULE 14: Intelligence Layer — Smart Staffing

Features

- Real-time staff recommendation (1:4 ratio)
- Hourly traffic heatmap (8AM-9PM, color-coded)
- Peak/quiet hour identification
- Server performance comparison
- Staffing recommendations

#	Test Type	Test Case	Expected Result
14.1	Smoke	Click "Smart Staffing"	View renders with heatmap and recommendation
14.2	Unit	Staff calc: 8 active tables ÷ 4 = 2 servers	Returns 2
14.3	Unit	Staff calc: 0 active tables → min 1	Returns 1 (never 0)
14.4	Unit	Peak hour detection (≥70% of max)	Correct hours flagged
14.5	Unit	Quiet hour detection (≤30% of max, >0)	Correct hours flagged
14.6	UAT	Payments concentrated at 12PM-2PM	Heatmap shows red at those hours
14.7	UAT	Top performer identification	Highest revenue server named

MODULE 15: Intelligence Layer — Customer Insights

Features

- Average dwell time (closed tables)

- Average spend per customer
- Repeat customer rate (same phone)
- Table utilization percentage
- Popular items ranking with visual bars
- Behavior pattern analysis

#	Test Type	Test Case	Expected Result
15.1	Smoke	Click "Customer Insights"	View renders with 4 metric cards
15.2	Unit	Avg dwell: (closedAt - openedAt) / closedTables	Correct minutes
15.3	Unit	Repeat rate: phones with >1 payment ÷ total unique phones	Correct %
15.4	Unit	Utilization: activeTables ÷ totalTables × 100	Correct %
15.5	Unit	Popular items sorted by qty descending	Top 8, correct order
15.6	UAT	5 closed tables → view dwell time	Realistic average shown
15.7	UAT	Same phone pays 3 times → repeat rate reflects	Shows 100% if only customer
15.8	UAT	No data state	Shows 0m, 0 spend, 0% gracefully

MODULE 16: Intelligence Layer — Anomaly Detection

Features

- 6 monitors: Low tips, Walkout risk, Revenue drops, Large payments, Staff disparity, Extended table times
- Severity levels: high, medium, low
- All-clear state when no issues
- Real-time monitoring status dashboard

#	Test Type	Test Case	Expected Result
16.1	Smoke	Click "Anomaly Detection"	View renders (either all-clear or issues)
16.2	Unit	3+ tables with 0 tips → "Low Tipping Pattern"	Medium severity anomaly generated
16.3	Unit	Table open 2h+ \$0 paid → "Walkout Risk"	High severity anomaly generated
16.4	Unit	Recent 3h revenue < 50% of prior 3h → "Revenue Drop"	Medium severity anomaly
16.5	Unit	Payment 3x above average → "Unusually Large"	Low severity anomaly
16.6	Unit	Server tip avg < 30% of max → "Tip Disparity"	Low severity anomaly
16.7	Unit	Closed table dwell > 2x avg → "Extended Table Times"	Low severity anomaly
16.8	Unit	No anomalies triggered	Shows All Clear
16.9	Unit	Anomalies sorted: high → medium → low	Correct order
16.10	UAT	Simulate walkout (open table 2h+)	Red "high" badge, walkout detail shown
16.11	UAT	All operations normal	Green "All Clear" badge

CROSS-CUTTING CONCERNS

#	Test Type	Test Case	Expected Result
X.1	Smoke	localStorage persistence	Refresh page → tables, catalogue, invoices intact
X.2	Smoke	Route <code>/pay</code> bypasses sidebar	No sidebar/nav wrapper
X.3	Smoke	Route <code>/table</code> bypasses sidebar	No sidebar/nav wrapper
X.4	Unit	Audio notification (Web Audio API)	Creates AudioContext, plays 880Hz for 0.3s

#	Test Type	Test Case	Expected Result
X.5	Unit	Web Share API fallback	Falls back to clipboard if navigator.share unavailable
X.6	UAT	Offline support (no network)	App loads from cache, localStorage data available
X.7	UAT	Dark mode	All views readable in dark theme
X.8	UAT	Performance: 50+ tables	No lag in overview, scroll smooth
X.9	UAT	Concurrent tab usage	localStorage syncs across tabs (storage event)

SUMMARY FOR DEVELOPERS

Total Test Cases: 132

Category	Smoke	Unit	UAT	Total
Navigation & Home	2	2	2	6
Invoice Management	2	14	6	22
Tap & Go POS	2	3	2	7
Customer Pay (/pay)	2	4	3	9
Table Service	2	9	5	16
Catalogue	1	5	2	8
Customer Table (/table)	2	8	10	20
AI Insights	1	4	3	8
Wallet & Reconciliation	1	1	2	4
Scan & Decode	1	3	1	5
Tips Analytics	1	2	2	5
Payment History	1	3	1	5
Revenue Forecast	1	4	2	7
Smart Staffing	1	4	2	7
Customer Insights	1	4	3	8
Anomaly Detection	1	7	2	10
Cross-Cutting	3	2	4	9
TOTAL	25	69	52	146

Implementation Priority

- P0 — Critical (Week 1):** - All Smoke tests (25) — ensures nothing is broken - Payment flows unit tests (Modules 3, 4, 7) — money handling correctness - Table lifecycle unit tests (Module 5) — core business logic
- P1 — High (Week 2):** - Invoice management unit tests (Module 2) - Split payment & tip calculations (Module 7) - Intelligence Layer unit tests (Modules 13-16) — ensures AI logic is correct
- P2 — Medium (Week 3):** - All UAT tests — requires test environment with localStorage seeded - Cross-cutting concerns

Recommended Testing Tools

Layer	Tool	Why
Unit	Vitest	Already in Vite ecosystem, fast, TS-native
Component	React Testing Library	Renders components in isolation
E2E/UAT	Playwright	Cross-browser, mobile viewport, real browser testing

Layer	Tool	Why
Visual	Chromatic / Percy	Catch CSS regressions
Smoke	Playwright smoke suite	Fast CI gate (< 2 min)

Key Test Data Requirements

- 1. **Seed tables:** 5 tables with mixed statuses (open, partially-paid, closed)
- 2. **Seed payments:** 15+ payments across tables with varying times, tips, servers
- 3. **Seed catalogue:** 10 items across 4 categories
- 4. **Seed invoices:** 10 invoices (mix of paid, pending, overdue, partial)
- 5. **Time manipulation:** Use `vi.useFakeTimers()` for walkout alerts, dwell time, auto-close tests

Architecture Notes for Testing

```
src/
├── __tests__/
│   ├── unit/
│   │   ├── invoice-logic.test.ts      (Module 2 pure functions)
│   │   ├── table-logic.test.ts        (Module 5 pure functions)
│   │   ├── split-calculations.test.ts (Module 7 split/tip math)
│   │   ├── ai-scoring.test.ts         (Module 8 AI functions)
│   │   └── intelligence.test.ts        (Modules 13-16 analytics)
│   ├── component/
│   │   ├── TapGoPOS.test.tsx
│   │   ├── TableServiceView.test.tsx
│   │   ├── InvoiceCreator.test.tsx
│   │   └── CustomerPayFlows.test.tsx
│   └── e2e/
│       ├── smoke.spec.ts              (All 25 smoke tests)
│       ├── payment-flows.spec.ts       (Customer pay + table pay)
│       ├── merchant-tables.spec.ts     (Full table lifecycle)
│       └── intelligence.spec.ts         (AI views render correctly)
├── test-utils/
│   ├── seed-data.ts                   (Factory functions for test data)
│   └── localStorage-mock.ts           (Mock localStorage for Vitest)
```

Definition of Done

- [] All 25 smoke tests passing in CI
- [] All 69 unit tests passing with >90% branch coverage on business logic
- [] All 52 UAT tests documented with manual test scripts
- [] No TypeScript errors (`tsc --noEmit clean`)
- [] Mobile viewport (375px) verified for all customer-facing routes
- [] localStorage persistence verified across browser refresh
- [] Audio notifications work (Web Audio API) on Chrome/Safari
- [] Dark mode renders correctly on all views